

Guía de estudio - Context API



¡Hola! Te damos la bienvenida a esta nueva guía de estudio.

¿En qué consiste esta guía?

La siguiente guía de estudio tiene como objetivo recordar y repasar los contenidos que hemos visto en clase, dentro de los que se encuentran:

- Compartir estados entre componentes utilizando props.
- Compartir estados entre componentes utilizando context.
- Conocer el flujo de trabajo de React Router con Context para compartir información entre múltiples páginas.

¡Vamos con todo!



Tabla de contenidos

Estado local	3
Modificando un estado desde el componente hijo	5
Prop Drilling	7
Context API	8
Implementado un contexto	8
¡Manos a la obra! - Mi primer Contexto	10
Setup inicial del proyecto	11
¡Manos a la obra! - Mi primer Contexto	12
Agregando contexto al ejercicio	12
1. Crear el contexto	12
2. Utilizar Provider	13
3. Consumir el contexto	13
¡Manos a la obra! - Revisión de un proyecto con contexto y rutas	15
¡Manos a la obra! - Intenta construir el proyecto desde cero	15
Preguntas de cierre	16



¡Comencemos!

Estado local

En React, cada componente tiene sus propios estados, los cuales no se comparten ni se pueden acceder directamente a ellos desde otro componente. Es por eso que decimos que los estados son locales.

Veamos un ejemplo:

Creemos un componente simple que incremente la cantidad de una cuenta cada vez que se hace clic, este lo podemos cargar fácilmente desde App.js en un proyecto nuevo para poder ejecutarlo. Esta componente se llamará Cuenta.jsx

```
1  import { useState } from "react"
2
3  const Cuenta = () => {
4    const [cuenta, setCuenta] = useState(0)
5
6    return (
7      <div>
8        <ul>
9          {cuenta}
10         <button onClick={() => setCuenta(cuenta + 1)}>+</button>
11       </ul>
12     </div>
13   )
14 }
15
16 export default Cuenta
17
```

Imagen 1. Código fuente de un componente con el estado cuenta
Fuente: Desafío Latam

Al ejecutar el proyecto en el navegador, veremos el texto 0 y un botón que lo incremente en 1 por cada clic.

Ahora intentemos dividir el proyecto en 2 componentes para demostrar que los estados son locales, para eso el botón lo transformaremos en una nueva componente llamado Aumentador.jsx

```
1 import { useState } from "react"
2 import Aumentador from "../Aumentador"
3
4 const Cuenta = () => {
5   const [cuenta, setCuenta] = useState(0)
6
7   return (
8     <div>
9       <ul>
10        {cuenta}
11        <Aumentador />
12      </ul>
13    </div>
14  )
15 }
16
17 export default Cuenta
18
19
```

Imagen 2. Componente padre donde se define el estado cuenta
Fuente: Desafío Latam

```
1 import { useState } from "react"
2
3 const Cuenta = () => {
4   const [cuenta, setCuenta] = useState(0)
5
6   return (
7     <div>
8       <ul>
9        {cuenta}
10        <button onClick={() => setCuenta(cuenta + 1)}>+</button>
11      </ul>
12    </div>
13  )
14 }
15
16 export default Cuenta
17
```

Imagen 3. Componente hija donde se define el estado cuenta
Fuente: Desafío Latam

Al ejecutar el proyecto de esta forma obtendremos error:

```
ERROR

[eslint]
src/Aumentador.jsx
  Line 6:28: 'setCuenta' is not defined  no-undef
  Line 6:38: 'cuenta' is not defined       no-undef
```

Imagen 4. Error dado que el estado es local
Fuente: Desafío Latam

Modificando un estado desde el componente hijo

Es posible pasar un estado y/o la función que lo modifica a un componente hijo, para eso simplemente los pasamos como props.

```
1 import { useState } from "react"
2 import Aumentador from "./Aumentador"
3
4 const Cuenta = () => {
5   const [cuenta, setCuenta] = useState(0)
6
7   return (
8     <div>
9       <ul>
10        {cuenta}
11        <Aumentador setCuenta={setCuenta} cuenta={cuenta} />
12      </ul>
13    </div>
14  )
15 }
16
17 export default Cuenta
```

Imagen 5. Pasando el estado local como prop
Fuente: Desafío Latam

Dentro de Aumentador.jsx recibimos setCuenta y cuenta:

```
1 import { useState } from "react"
2
3 const Aumentador = ({setCuenta, cuenta}) => {
4
5   return (
6     <button onClick={() => setCuenta(cuenta + 1)}></button>
7   )
8 }
9
10 export default Aumentador
```

Imagen 6. Recibiendo el estado local como prop
Fuente: Desafío Latam

Cuando la situación es al revés, o sea, originalmente el estado está en la componente hija y queremos que sea compartido con el componente padre, tenemos que mover el estado a esta y traspasarlo como props a la componente hija tal como aprendimos.

Importante:



- Los estados son locales al componente.
- Podemos compartir un estado con una componente hija pasándolo como prop.

Prop Drilling

En algunos casos, para poder separar el código en múltiples componentes tenemos que pasar el estado a lo largo de todas ellas, a esto se le conoce en la industria como prop drilling.

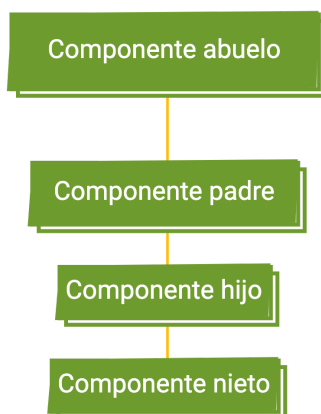


Imagen 7. Diagrama de traspaso de props a lo largo de múltiples componentes relacionados.

Fuente: Desafío Latam

Y la situación podría ser peor, es muy normal que un componente pueda tener varios componentes relacionados y podemos necesitar que todos estos modifiquen los mismos estados.

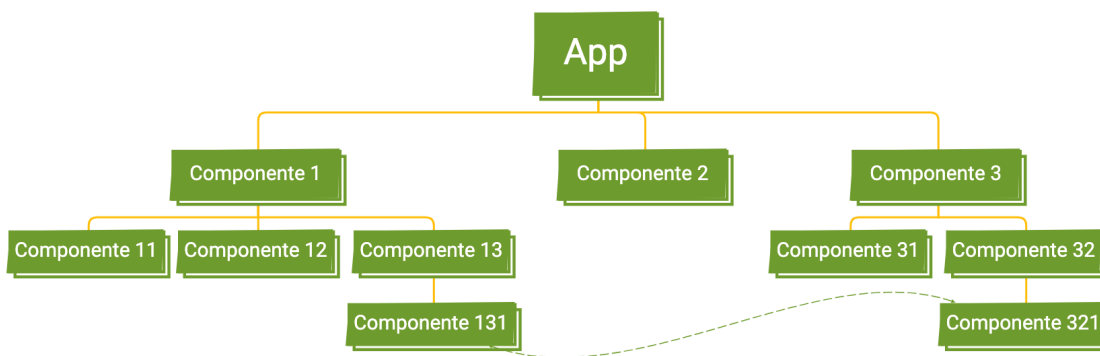


Imagen 8. Diagrama de traspaso de props a lo largo de múltiples componentes relacionados en un caso más complejo.

Fuente: Desafío Latam

Context API

En React podemos ocupar Context para crear un estado global o un estado de mayor alcance que un estado local para compartir estados entre múltiples o incluso todas las componentes.

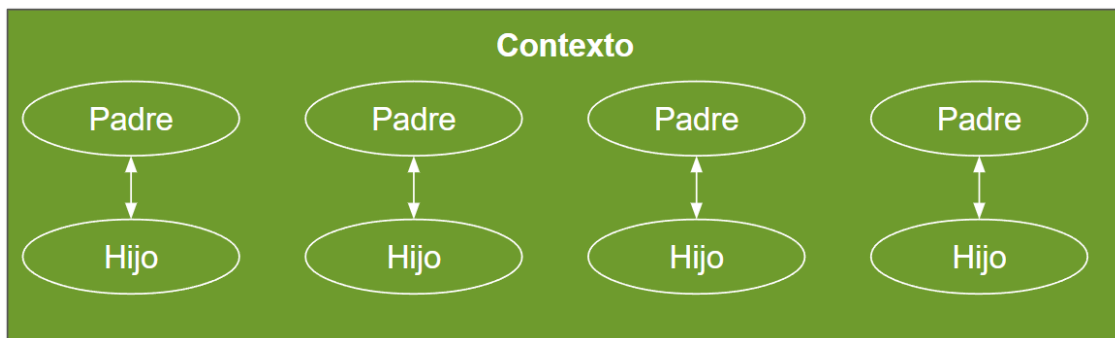


Imagen 9. Diagrama de Context
Fuente: Desafío Latam

Incluso los componentes hijos y demás también podrán acceder directamente a la misma fuente de datos centralizada.

Implementado un contexto

Al trabajar con contextos tendremos que:

1. Crear un contexto.
Documentación oficial: <https://react.dev/reference/react/createContext>
2. Proveer un contexto especificando los valores que vamos a compartir.
3. Consumir un contexto.

Estos pasos los repetiremos cada vez que agreguemos un nuevo contexto.

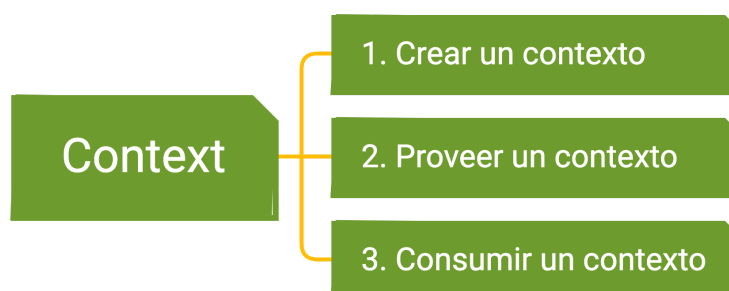


Imagen 10. Conceptos importantes de Context
Fuente: Desafío Latam

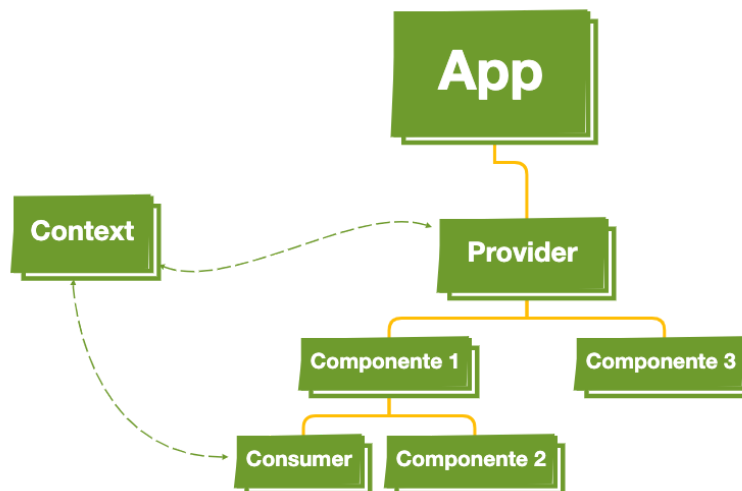


Imagen 11. Conceptos importantes de Context
Fuente: Desafío Latam

- Cada **Context** lo debemos crear como un módulo independiente que importamos donde lo necesitamos.
- Envolveremos todos los componentes que queremos mantener actualizados dentro del componente **Provider**.
- Cualquier componente puede ser un **Consumer**, cuando se modifique el valor de un contexto se volverá a renderizar el consumer.



¡Manos a la obra! - Mi primer Contexto

Realizaremos el ejercicio para mostrar un contador de clic.

- El componente 1 tendrá un botón para aumentar "Increment.jsx".
- El componente 2 tendrá un botón para disminuir "Decrement.jsx".
- Crearemos un **context** y un **provider** para hacer nuestro estado global.

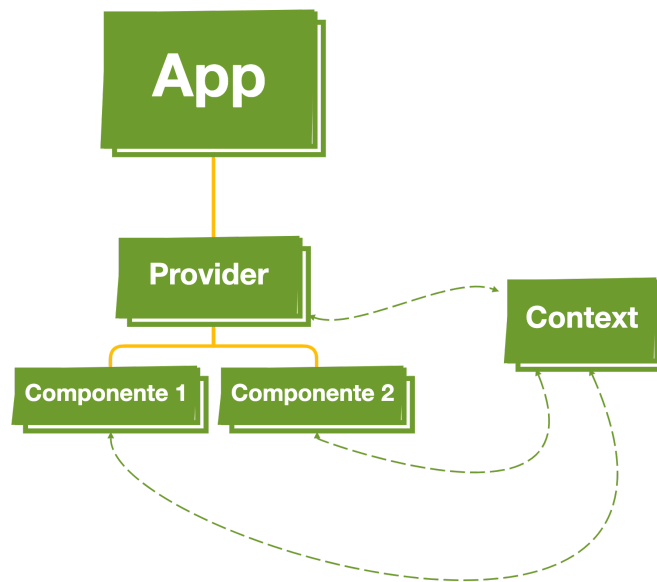


Imagen 12. Diagrama de componentes del ejercicio
Fuente: Desafío Latam

Setup inicial del proyecto

- Crearemos el proyecto con **npm create vite@latest name-proyect**
- Generaremos el componente "Increment.jsx"
- Crearemos el componente "Decrement.jsx"
- Importaremos ambos componentes dentro de "App.jsx"

src\components\Increment.jsx

```
const Increment = () => {  
  return (  
    <button onClick={() => {}}>  
      Increment: 0  
    </button>  
  );  
};  
export default Increment;
```

src\App.jsx

```
import Decrement from "../components/Decrement";  
import Increment from "../components/Increment";  
  
const App = () => {  
  return (  
    <div>  
      <Increment />  
      <Decrement />  
    </div>  
  );  
};  
export default App;
```



¡Manos a la obra! - Mi primer Contexto

Crea la componente 2 "Decrement.jsx", basándose en el componente "Increment.jsx"

Agregando contexto al ejercicio

1. Crear el contexto

Para crear un contexto modularizado, generaremos un archivo en la carpeta src llamado src\context\CounterContext.jsx . En su interior, escribamos el siguiente código:

```
import { createContext, useState } from "react";

export const CounterContext = createContext();

const CounterProvider = ({ children }) => {
  const [counter, setCounter] = useState(0);
  return (
    <CounterContext.Provider value={{ counter, setCounter }}>
      {children}
    </CounterContext.Provider>
  );
};

export default CounterProvider;
```

Esto nos permitirá disponibilizar para nuestra aplicación el contexto como un módulo que se pueda importar desde cualquier componente 🙌

```
export const CounterContext = createContext();
```

CounterContext tiene la propiedad **Provider** la cual expone los datos a todos los componentes anidados, es por esto que se le pasa el children en su interior.

Este Proveedor del contexto es el que contiene los datos globales que quieras compartir, en este caso exportamos el estado "counter" y su método "setCounter"

```
<CounterContext.Provider value={{ counter, setCounter }}>
```

```
{children}  
</CounterContext.Provider>
```

Es bueno recordar que value está recibiendo un objeto, el que tiene las propiedades y valores correspondientes a tus datos globales:

```
value={{ counter, setCounter }}
```

2. Utilizar Provider

Lo siguiente será modificar el código del archivo App.jsx para utilizar el provider.

- Debemos envolver nuestra aplicación en el componente Provider **<CounterProvider>**

```
import CounterProvider from "../context/CounterContext";  
  
import Decrement from "../components/Decrement";  
import Increment from "../components/Increment";  
  
const App = () => {  
  return (  
    <div>  
      <CounterProvider>  
        <Increment />  
        <Decrement />  
      </CounterProvider>  
    </div>  
  );  
};  
export default App;
```

3. Consumir el contexto

Para consumir el contexto, utilizamos useContext, es similar a useState solo que los valores los traemos del contexto en vez de ser locales al objeto

En el componente 1 **Increment.jsx** aumentamos la cuenta.

```
import { useContext } from "react";  
import { CounterContext } from "../context/CounterContext";
```

```
const Increment = () => {
  const { counter, setCounter } = useContext(CounterContext);

  return (
    <button onClick={() => setCounter(counter + 1)}>
      Increment: {counter}
    </button>
  );
};
export default Increment;
```

En el componente 2: **Decrement.jsx** disminuimos la cuenta.

```
import { useContext } from "react";
import { CounterContext } from "../context/CounterContext";

const Decrement = () => {
  const { counter, setCounter } = useContext(CounterContext);
  return (
    <button onClick={() => setCounter(counter - 1)}>
      Decrement: {counter}
    </button>
  );
};
export default Decrement;
```



¡Manos a la obra! - Revisión de un proyecto con contexto y rutas

Descarga el proyecto Planet Blog:

1. Descarga el proyecto de la plataforma:
 - a. En el terminal, dentro de la carpeta del proyecto, deberás ejecutar:
 - i. `npm install`
 - ii. `npm dev`
 - iii. Si tienes problemas con el proyecto, en la unidad anterior aparece un troubleshooting
 - b. Navega las secciones del proyecto, desde admin crea una publicación y observa la publicación creada desde el home.
2. Revisa el archivo App.jsx
 - a. ¿Qué componentes se cargan?
 - b. ¿Desde dónde se carga el contexto?
 - c. ¿Cuál es el nombre del contexto?
 - d. ¿Qué dependencias o componentes externos al proyecto son cargados?
 - i. El archivo package.json te puede dar pistas.
3. Revisa el componente Admin y los componentes que carga.
 - a. ¿Dónde se ocupa el contexto?
4. Revisa el componente Home y los componentes que carga.



¡Manos a la obra! - Intenta construir el proyecto desde cero

¡Ahora te toca a ti!, crea la misma aplicación desde cero.

Preguntas de cierre

Reflexiona:

- ¿Cómo podemos pasar un estado de un componente hijo?
- ¿En qué consiste el problema del prop drilling?
- ¿Para qué sirve Context?
- ¿Cuál es el rol del componente Provider?
- ¿Para qué sirve useContext?
- ¿Qué sucede con un consumer cuando el valor entregado a Provider cambia?
- ¿Siempre es mejor utilizar context?



s